

DEV2 – JAVL – Laboratoire Java**TD 09 – Héritage****Résumé**

Dans ce TD, nous allons explorer la notion d'héritage. Cette notion est un concept fondamental de la programmation orientée objet permettant la création de nouvelles classes basées sur des classes existantes. Cette approche permet de réutiliser du code en héritant des attributs et des méthodes de la classe parent tout en permettant aux classes dérivées d'étendre ou de spécialiser leur comportement.

Table des matières

1	Les animaux	2
1.1	Le chat	2
1.2	La classe animal	2
1.3	Override	4
1.4	Equals	4
1.5	La classe Object	6
2	Les véhicules	6

1 Les animaux

1.1 Le chat

Nous allons débiter notre exploration de la notion d'héritage en se basant sur les classes `Dog` et `Cat` que nous avons construites lors des premiers TDs.

Tout comme les chiens, nos chats seront caractérisés par leur nom, leur poids, ainsi que tous les autres attributs déjà présents dans la classe `Dog`.

Exercice 1

Ajuster Cat

Ajustez votre classe `Cat` pour refléter ces similitudes, en ajoutant ce qui manque.

La seule différence qui doit subsister est que les chats n'aboient pas : ils miaulent.

Profitez-en et mettez ces classes (et la classe `Main`) dans un package `g12345.pets`.

Cette façon de faire n'est pas la plus pratique, car elle vous demande de dupliquer une grande quantité de code. De plus, si vous devez changer une des méthodes de la classe `Dog`, il faudra changer à l'identique la méthode de la classe `Cat` si l'on veut que les comportements des deux animaux restent identiques. Pour surmonter ces désagréments, nous allons introduire une nouvelle classe et la notion d'héritage dans la suite de ce TD.

1.2 La classe animal

Créez maintenant une nouvelle classe `Animal`, copiez-y tous les attributs et les méthodes de la classe `Dog` à l'exception de la méthode `bark`, et adaptez le nom du constructeur en `Animal`. Pour le moment, cette nouvelle classe fait exactement la même chose que nos chats et nos chiens à l'exception qu'elle ne peut ni aboyer ni miauler. Dans une classe `Main`, on peut créer quelques chiens, chats, et même un animal.

```
public class Main {
    public static void main(String[] args) {
        Animal rex = new Animal("Rex", 10);
        Dog medor = new Dog("Medor", 60);
        Cat felix = new Cat("Felix", 5);
        rex.run();
        medor.run();
        felix.run();
    }
}
```

Nous allons rendre la classe `Animal` le parent des classes `Dog` et `Cat`. Ceci nous permettra de ne pas avoir à ré-écrire dans `Dog` et `Cat` toutes les méthodes et les attributs que nous avons déjà écrits dans `Animal`. Mais avant de pouvoir faire cela, nous allons devoir effectuer un petit changement dans la visibilité des attributs de la classe `Animal`.

Comme discuté dans des TDs précédents, devant certains de vos attributs ou de vos méthodes, vous écriviez parfois des mots comme `public` ou `private` alors que parfois vos méthodes et attributs ne sont précédés de rien. Il s'agit de la visibilité de vos attributs et méthodes. Voici un petit rappel des différentes visibilités.

Visibilité La visibilité permet de savoir qui peut avoir accès et donc utiliser l'attribut ou la méthode. Il existe plusieurs visibilité en java :

- ▷ **public** : toutes mes classes peuvent utiliser l'attribut ou la méthode.
- ▷ **private** : la méthode ou l'attribut ne peut être accédé que dans la classe à laquelle il appartient.
- ▷ **protected** : seules les classes dans le même package et les classes qui héritent de la classe où la méthode ou l'attribut est défini peuvent accéder à la méthode ou à l'attribut.
- ▷ par défaut (quand rien n'est mentionné) : seules les classes dans le même package peuvent accéder à la méthode ou à l'attribut.

Commencez par changer la visibilité de tous les attributs de la classe **Animal** en **protected**. C'est maintenant que la magie va opérer. Supprimez toutes les méthodes de la classe **Dog** à l'exception de la méthode **bark**. Ajoutez ensuite devant le nom de la classe : **extends Animal**, et ré-écrivez le constructeur comme suit :

```
public class Dog extends Animal{  
  
    Dog(String name, int weight) {  
        super(name, weight);  
    }  
    void bark() {  
        // ...  
        // votre méthode bark  
    }  
}
```

Plusieurs mots-clés apparaissent ici, d'abord le **extends** qui va signifier que la classe **Animal** est la **classe parent** de la classe **Dog**. De façon équivalente, on dit que la classe **Dog** **hérite** de la classe **Animal**, ou encore que la classe **Dog** est **l'enfant** de la classe **Animal**. Cela signifie que la classe **Dog** possède également toutes les méthodes définies dans **Animal** et qu'elle possède également tous ses attributs.

Le constructeur a également changé, on appelle une méthode **super**. Ce mot-clé **super** fait référence à la classe parent, quand on utilise **super(name, weight)** on fait même référence au constructeur de la classe parent. Il va donc appeler le constructeur de la classe **Animal**.¹

Essayez de relancer votre main. Vous constatez qu'il continue de fonctionner même s'il n'y a plus de méthode **run** à l'intérieur de **Dog**. En effet, comme notre chien hérite de la classe **Animal** il a automatiquement toutes les méthodes de celle-ci.

Exercice 2

Héritage

Commencez par faire de même pour la classe **Cat**, elle doit aussi hériter de la classe **Animal**. Ensuite essayez les différentes modifications suivantes :

- ▷ Essayez de supprimer le constructeur de **Dog**, et de remettre son ancien constructeur, est ce que cela fonctionne ? Pourquoi ?

1. Vous avez déjà rencontré un mot-clé assez similaire au mot-clé **super**, il s'agit du mot-clé **this**. Celui-ci désigne l'instance actuelle de la classe, c'est-à-dire l'objet. Vous l'avez notamment déjà utilisé dans des constructeurs pour lever l'ambiguïté entre le paramètre du constructeur et l'attribut de l'objet, par exemple : **this.name = name**.

- ▷ Changez dans la classe `Animal` la visibilité des attributs en `private`, est-ce que ça fonctionne ? Pourquoi ?
- ▷ Écrivez un constructeur `Dog(String name, int weight, boolean hungry)` qui fonctionne, sans redéfinir un constructeur dans la classe `Animal` (utilisez le mot-clé `super` pour accéder à l'attribut `hungry` de la classe `Animal`)

1.3 Override

L'héritage nous a permis d'écrire une seule fois toutes nos méthodes et nos attributs, mais imaginons maintenant que l'on veuille modifier le comportement de la méthode `run()` pour nos chats. En effet, ceux-ci sont plus agiles et ne craignent pas de se blesser quand ils courent. C'est possible de ré-écrire la méthode `run()` dans la classe `Chat`, bien qu'elle existe déjà dans la classe parent. Il est fortement conseillé d'utiliser le mot-clé `@Override` devant la fonction `run()` de la classe `Cat`. Ce mot-clé permet notamment de s'assurer que la fonction que vous redéfinissez existe bien dans une classe parent, il favorise aussi une bonne lisibilité du code.

Exercice 3 Override la course du chat

Ré-écrivez la méthode `run()` dans la classe `Chat` en utilisant le mot-clé `@Override` pour que le chat ne risque pas de se blesser quand il court.

```
class Cat extends Animal{
    //... votre constructeur et la méthode meow
    @Override
    public run(){
        //... votre méthode run() modifiée
    }
}
```

Exercice 4 Le son du miaulement

Ré-écrivez la méthode `sound()` dans la classe `Chat` en utilisant le mot-clé `@Override` pour que la méthode soit en accord avec la méthode `meow()`.

1.4 Equals

Admettons maintenant que nous voulions que deux animaux soient égaux s'ils ont exactement les mêmes attributs. Comment pourrait-on faire cela ?

Commencez par créer deux chiens `dog` et `dog2` avec le même nom et le même poids. Et essayez le code suivant dans votre `main` :

```
System.out.println(dog == dog2);
System.out.println(dog.equals(dog2));
```

Vous constatez que les deux lignes affichent `false`. Même s'ils ont des attributs identiques, Java les considère comme différents. La méthode `equals`, est une méthode que toutes les classes possèdent, nous verrons d'où elle vient plus tard. Elle compare les adresses mémoires des deux objets, qui forcément sont différentes.

Dans la classe `Animal`, redéfinissez la méthode `equals` pour qu'elle compare les attributs des deux animaux. Pour faire cela, appuyez sur le click droit de votre souris et choisissez `Generate hashCode() and equals()`. Lors de la première génération par IntelliJ du `hashCode()` vous devriez avoir une fenêtre comme celle-ci qui s'ouvre.

Choisissez le même template que celui sur l'image et appuyer sur Next. IntelliJ vous demande quels attributs il faut prendre, sélectionnez les tous. IntelliJ vous demande

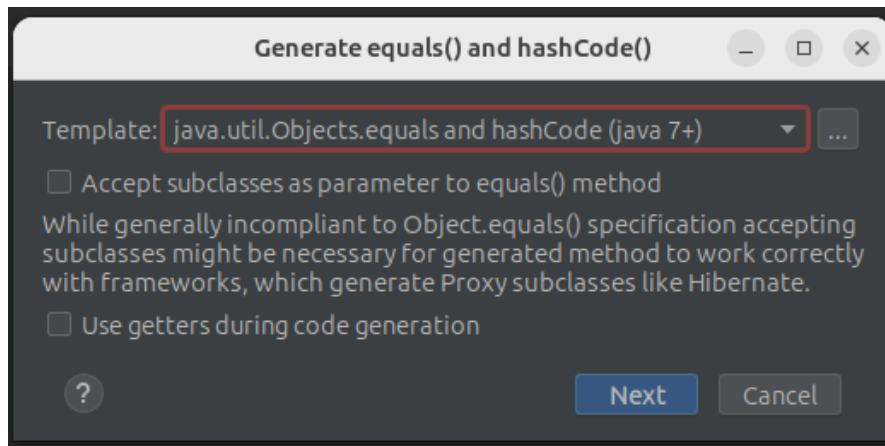


FIGURE 1 – Première génération du hashCode

également si vous voulez sélectionner le champs non-null, sélectionnez le également. Vous devriez obtenir quelque chose comme ceci :

```
@Override
public boolean equals(Object o) {
    if (this == o) return true;
    if (o == null || getClass() != o.getClass()) return false;
    Animal animal = (Animal) o;
    return poids == animal.poids
        && hungry == animal.hungry
        && injured == animal.injured
        && Objects.equals(name, animal.name);
}

@Override
public int hashCode() {
    return Objects.hash(name, poids, hungry, injured);
}
```

La première méthode commence par regarder si l'objet est le même que celui sur lequel on appelle la méthode, si c'est le cas, on retourne `true`. On regarde si l'objet est `null` ou s'il n'est pas de la même classe que l'objet sur lequel on appelle la méthode, si c'est le cas, on retourne `false`. Ensuite, on caste l'objet en `Animal`. Nous aborderons la notion de **casting** dans le TD suivant, ceci permet de transformer certains types de variable en d'autres. Il est essentiel de le faire ici, car la variable reçue en paramètre est un `Objet` (nous discuterons de ce type à la section suivante) et non `Animal`, si on n'utilise pas le casting, on ne peut donc pas accéder aux attributs comme `name` ou `weight` puisque le type `Objet` ne possède pas ces attributs. Nous reviendrons sur cette notion lors du TD suivant. Finalement, on compare les attributs des deux animaux, s'ils sont tous égaux il retournera `true`, sinon `false`. Réessayez le code précédent écrit dans le main, vous constaterez que la deuxième ligne affiche maintenant `true`.

La méthode `hashCode` est une méthode qui permet de générer un code unique pour chaque objet, elle est utilisée par certaines structures de données, comme les `HashMap` pour stocker les objets. Elle a été ici modifiée pour qu'elle génère un code unique pour chaque animal ayant les mêmes attributs. Deux objets "equals" doivent avoir le même "hashCode", c'est pour cette raison que ces deux méthodes sont générées en même temps.

1.5 La classe Object

En fait, en Java toutes les classes, y compris celles que vous avez écrites, héritent d'une classe appelée `Object`. C'est dans cette classe que se trouvent les méthodes `equals` et `hashCode`.

Il existe une autre méthode de cette classe qu'il est possible de redéfinir, il s'agit de la méthode `toString`. Cette méthode est appelée quand on fait un `System.out.println`. Vous avez remarqué que quand vous avez fait un `System.out.println(dog)`, il a affiché quelque chose comme `Dog@1b6d3586`, nous pouvons redéfinir cette méthode pour qu'elle affiche le nom et le poids de l'animal. Pour faire cela, appuyez sur le click droit de votre souris et choisissez `Generate toString()`. Essayez cette méthode sur un de vos chiens dans votre main. Modifiez ensuite cette méthode `toString()` pour qu'elle permette d'obtenir un affichage comme suit : `Medor: poids = 60kg`

2 Les véhicules

Exercice 5 classe Vehicule

- ▷ Créez un nouveau package `g12345.motorizedvehicle`. Dans celui-ci, créez une classe `Vehicle` qui possède les attributs suivants : la marque de la voiture `String brand`, la capacité maximum du réservoir `double maxTank` et la capacité actuelle du réservoir `double tank`.
- ▷ Générez le constructeur de cette classe avec deux paramètres, `String brand` et `double maxTank`. L'attribut `tank` sera initialisé à la valeur de `maxTank`.
- ▷ Créez une méthode `void fillUpGaz()` qui remet le réservoir (`tank`) à sa valeur maximale.
- ▷ Créez une méthode `ride` qui prend en paramètre un entier `distance` et qui affiche "Je roule sur `distance` km". Pour chaque km parcouru, le réservoir diminue de 0.1L (c'est-à-dire de 0.1 unité). Si le réservoir est vide, la méthode affiche "Je suis en panne sèche".
- ▷ Générez les méthodes `equals`, `hashCode`

Exercice 6 classe Moto

- ▷ Créez une classe `Moto` qui hérite de la classe `Vehicle`.
- ▷ Générez le constructeur de cette classe.
- ▷ Générez la méthode `toString`. Elle devra afficher la marque, la capacité maximum du réservoir, ainsi que la capacité actuelle du réservoir.

Exercice 7 la classe Voiture

- ▷ Créez une classe `Car` qui hérite de la classe `Vehicle`. Elle possède un attribut `int nSeats` qui représente le nombre de places.
- ▷ Générez le constructeur de cette classe. Ce constructeur prendra en paramètre la marque, la capacité maximum du réservoir et le nombre de places.
- ▷ Générez la méthode `toString`. Elle devra afficher la marque, le nombre de places, la capacité maximum du réservoir, ainsi que la capacité actuelle du réservoir.

Exercice 8 la classe Voiture Hybride

- ▷ Créez une classe `HybridCar` qui hérite de la classe `Car`.
- ▷ Générez le constructeur de cette classe.
- ▷ Ré-écrivez la méthode `ride` pour qu'elle diminue le réservoir seulement de 0.05L par km parcouru.

Exercice 9 la classe Main

- ▷ Créez une classe `Main` qui crée une moto, une voiture et une voiture hybride.
- ▷ Faites rouler la moto, la voiture et la voiture hybride.